

Лабораторная работа № 4

Бинаризация и морфологические преобразования изображений

1. Бинаризация изображений

Бинарное изображение – это цифровое изображение, пиксели которого имеют значения 0 или 1. При этом значения 1 соответствуют пикселям переднего плана (объектам), а 0 – пикселям фона.

Бинарные изображения используются в качестве входных данных для алгоритмов обработки, разработанных с учетом особенностей решаемой задачи, в ряде приложений, например, при обработке документов и в некоторых задачах промышленного машинного зрения.

Для формирования бинарного изображения по данным полутонового или цветного изображения можно выполнить операцию, которая выбирает некоторое подмножество пикселей изображения в качестве пикселей переднего плана. Эти пиксели представляют интерес для решаемой задачи анализа изображений. Остальные пиксели игнорируются как фоновые.

Операция отбора пикселей может быть простой, например, пороговый оператор, который отбирает пиксели со значениями из заданного яркостного диапазона или цветового подпространства, или представлять собой сложный алгоритм классификации.

1. 1. Пороговая бинаризация

Бинарные изображения можно получить из полутоновых изображений посредством пороговой бинаризации. Зная распределение значений яркости на данном изображении, некоторые значения можно выбрать в качестве порогов, разделяющих пиксели на группы. В простейшем случае выбирается одно пороговое значение t . Все пиксели с яркостью больше или равной t становятся пикселями переднего плана, а остальные — фоновыми. Эта операция называется *верхней пороговой бинаризацией*. При выполнении *нижней пороговой бинаризации* пикселями переднего плана становятся пиксели с яркостью меньше или равной t . При *пороговой бинаризации по диапазону* указываются два пороговых значения — верхнее и нижнее.

Пиксели с яркостью в диапазоне между пороговыми значениями выбираются в качестве пикселей переднего плана. Обратный критерий выбора пикселей переднего плана применяется при выполнении *внешней пороговой бинаризации*. Главным вопросом, возникающим при выполнении всех этих простейших разновидностей операции бинаризации, является выбор пороговых значений.

В библиотеке OpenCV для пороговой бинаризации используется функция *cvThreshold*.

```
double cvThreshold(  
    CvArr*      src,           /*Исходный массив изображения*/  
    CvArr*      dst,           /*Массив полученного бинарного изображения*/  
    double      threshold,     /*Пороговое значение*/  
    double      max_value,     /*Максимальное значение яркости пикселя*/  
    int         threshold_type /*Тип задаваемого порога*/  
);
```

1. 2. Адаптивная бинаризация

Если яркость изображения сильно зависит от положения пикселей, например, изображение в целом светлее в левом верхнем углу и темнее в правом нижнем, то больший смысл может иметь использование локальных порогов бинаризации для различных фрагментов изображения. Этот подход иногда называется *динамической пороговой бинаризацией*. В этом случае порог бинаризации вычисляется отдельно для каждого пиксела.

Для осуществления динамической (адаптивной) бинаризации изображения в OpenCV имеется функция *cvAdaptiveThreshold*.

```
void cvAdaptiveThreshold(  
    const CvArr* src,           /* Исходное изображение*/  
    CvArr* dst,                 /*Конечное изображение*/  
    double max_value,           /*Максимальное значение яркости*/  
    int adaptive_method,        /*Метод адаптивной бинаризации*/  
    int threshold_type,         /*Тип порога бинаризации*/  
    int block_size=3,           /*Размер окрестности пикселя*/  
    double param1=5             /*Параметр, зависящий от метода бинаризации*/  
);
```

Можно задать один из двух типов порога бинаризации:

1. *CV_THRESH_BINARY*

$$dst(x, y) = \begin{cases} max_value, & \text{если } src(x, y) > T(x, y), \\ 0 & \text{в остальных случаях} \end{cases} \quad (1)$$

2. *CV_THRESH_BINARY_INV*

$$dst(x, y) = \begin{cases} 0, & \text{если } src(x, y) > T(x, y), \\ max_value & \text{в остальных случаях} \end{cases} \quad (2)$$

При этом порог для каждого пиксела $T(x,y)$ может вычисляться следующими способами.

1. *CV_ADAPTIVE_THRESH_MEAN_C*: $T(x,y)$ - среднее из $block_size$ * $block_size$ соседних пикселей за вычетом константы $param1$.

2. *CV_ADAPTIVE_THRESH_GAUSSIAN_C*: $T(x,y)$ - это весовая сумма (Гауссиан) из $block_size * block_size$ соседних пикселей за вычетом константы $param1$.

2. Морфологические преобразования изображения

При осуществлении морфологического преобразования изображения выбирается структурообразующее множество любого размера и любой формы и центр этого множества. Над пикселями множества осуществляется определенное преобразование, и его результат присваивается значению центрального пикселя множества. По умолчанию в функциях OpenCV структурообразующее множество имеет размеры 3x3 и центр в середине множества, «крест» (рис. 1).

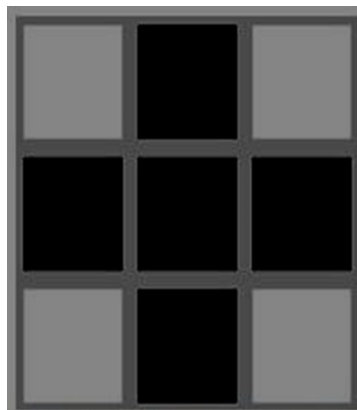


Рис. 1. Структурирующий элемент в виде креста

2.1. Типы морфологических операций

1. **Эрозия объекта (erode)** приводит к замене значений граничных пикселей объекта на 0. Однократное применение эрозии приводит к удалению слоя границы толщиной в 1 пиксель.

$$erode(x, y) = \min_{(x', y') \in kernel} src(x + x', y + y') \quad (3)$$

Последовательность преобразований: $dst = erode(src)$.

Для осуществления эрозии в OpenCV имеется функция `cvErode`.

```
void cvErode(  
IplImage* src,           /*Исходное изображение*/  
IplImage* dst,           /*Конечное изображение*/  
IplConvKernel* B = NULL, /*Структурирующее множество*/  
int iterations = 1       /*Количество повторений операции*/  
);
```

2. **Нарращение объекта (dilate)** приводит к замене значений пикселей фона, граничащих с объектом, на 1. Однократное применение наращивания приводит к добавлению к объекту слоя толщиной в 1 пиксель.

$$dilate(x, y) = \max_{(x', y') \in kernel} src(x + x', y + y') \quad (4)$$

Последовательность преобразований: $dst = dilate(src)$.

Для осуществления дилатации в OpenCV имеется функция `cvDilate`.

```
void cvDilate(  
IplImage* src,           /*Исходное изображение*/  
IplImage* dst,           /*Конечное изображение*/  
IplConvKernel* B = NULL, /*Структурирующее множество*/  
int iterations = 1       /*Количество повторений операции*/  
);
```

3. **Размыкание (open)** приводит к соединению областей фона, ранее разъединенных узкими участками пикселей объектов.

$$open(src) = dilate(erode(src)).$$

4. **Замыкание (close)** приводит к удалению небольших по площади фрагментов фона внутри объектов, например, "дыр" (замкнутых областей фона внутри объекта).

$$close(src)=erode(dilate(src)).$$

5. Морфологический градиент (morph_grad) приводит к выделению периметра областей яркости.

$$morph_grad(src)=dilate(src)-erode(src)$$

6. «Верх шляпы (tophat)» соответствует вычитанию из исходного изображения результата его размыкания.

$$tophat(src)=src-open(src)$$

7. «Чёрная шляпа» (blackhat) соответствует вычитанию из замыкания изображения самого изображения.

$$blackhat(src)=close(src)-src$$

Для осуществления морфологических операций в библиотеке OpenCV можно использовать функцию **cvMorphologyEx**.

```
void cvMorphologyEx(
const CvArr* src,           /*Входное изображение*/
CvArr* dst,                 /*Выходное изображение*/
CvArr* temp,                /*Буферное изображение*/
IplConvKernel* element,     /*Структурирующее множество*/
int operation,              /*Тип операции*/
int iterations;             /*Количество применений операции*/
);
```

Могут применяться и другие структурирующие элементы (рис. 2).



Прямоугольник

Диск

Кольцо

Рис. 2. Структурирующие элементы в виде прямоугольника, диска и кольца

Для получения структурирующего элемента может быть использована функция:

```
IplConvKernel* cvCreateStructuringElementEx (
```

```

int cols,
int rows,
int anchor_x,
int anchor_y,
int shape,
int* values = NULL);

```

Она возвращает указатель на структуру `IplConvKernel`, описывающую структурирующий элемент. Аргументы функции:

`cols` – число столбцов

`rows` – число столбцов и строк,

`anchor_x` – столбец элемента, для которого выполняется операция (значение `-1` означает средний столбец),

`anchor_y` – строка элемента, для которого выполняется операция (значение `-1` означает среднюю строку),

`shape` – флаг структурирующего элемента:

`CV_SHAPE_RECT` – прямоугольный структурирующий элемент,

`CV_SHAPE_ELLIPSE` – эллиптический структурирующий элемент,

`CV_SHAPE_CROSS` – крестообразный структурирующий элемент,

`CV_SHAPE_CUSTOM` – пользовательский структурирующий элемент,

`values` – указатель на растровое представление структурирующего элемента размерами `cols` × `rows` (при значении `shape`, равном `CV_SHAPE_CUSTOM`, при остальных значениях `shape` игнорируется и может быть равен `NULL`).

Освободить память, выделенную при создании структурирующего элемента, можно посредством функции `cvReleaseStructuringElement`.

Табл. 1. Типы морфологических операций

Тип операции	Морфологический оператор	Необходимость temp
CV_MOP_OPEN	Открытие	Нет
CV_MOP_CLOSE	Закрытие	Нет
CV_MOP_GRADIENT	Морфологический градиент	Да
CV_MOP_TOPHAT	«Верх шляпы»	Только если src=dst
CV_MOP_BLACKHAT	«Чёрная шляпа»	Только если src=dst

3. Задание

Реализовать программу, осуществляющую бинаризацию и морфологическое преобразование, указанные в варианте задания.

1. Пороговая бинаризация, эрозия, структурирующий элемент: крест.
2. Адаптивная бинаризация, дилатация, структурирующий элемент: диск.
3. Пороговая бинаризация, размыкание, структурирующий элемент: крест.
4. Адаптивная бинаризация, замыкание, структурирующий элемент: диск.
5. Пороговая бинаризация, морфологический градиент, структурирующий элемент: крест.
6. Пороговая бинаризация, эрозия, структурирующий элемент: диск.